A FROTHY V0.1 FIELD GUIDE

Frothy From the Ground Up

A two-day tour through the language, the runtime, and the image that makes it feel alive.

FROTHY PROJECT · EDITION 2026.04

Contents

Learn Frothy in 30 Minutes

Quick Syntax Map
Literals and direct values
Stable top-level slots
Rebinding is central
Functions, anonymous code, and computed calls
Locals, mutation, and block results
Conditionals
Boolean short-circuiting
repeat and while
Cells: Frothy's narrow array value
Records
Prefix groups with in
Inspection is part of normal work
Persistence: save, restore, and dangerous.wipe
Boot hooks and safe boot
One small board and FFI example
Common patterns from other languages
Edge cases worth learning early

How to Read This Guide in Two Days

Part I: Using Frothy

Frothy's Core Mental Model and What Changed from Froth
The Runtime Value Model: What a Value Is in Frothy
Stable Top-Level Slot Identity, Base Image vs Overlay Image, and Rebinding Semantics
Running Frothy in Practice
Frothy Projects in Practice
Build and Flash Workflows

Part II: How Frothy Works

Memory, Ownership, Lifetimes, Refcounts, Capacities, Cellspace Usage, and Reset Stories
Surface Syntax, Parser Structure, Lexical Resolution, and Why Code Is Non-Capturing
Canonical IR: Node Types, Literals, Links, Rendering, and Why IR Is the Persisted Code Truth
Evaluator Architecture: Frames, Locals, Slot Lookup, Calls, Mutation, Loops, Interrupts, and Safe Points
Cells: Why They Are Narrow, What They Can Store, How Indexing Works, and Why They Are Top-Level Owned
Persistence: save, restore, dangerous.wipe, Snapshot Layout, Symbol/Object Tables, Pointer-Free Restore, and Failure Handling
Inspection: words, show, core, info, and How Inspection Reflects Canonical IR Rather Than Raw Source
Interactive Profile: Boot, Safe Boot, REPL, Multiline Input, Prompt Behavior, Ctrl-C, Recovery
Control Transport: .control Mode, Single-Owner Sessions, Framed Requests/Events, Helper/Editor Story

Part III: Extending Frothy

FFI Boundary: Native Bindings, Board Slots, Allowed Types, Non-Persistable State, Substrate Reuse
Creating Project FFI in Practice
Creating Board FFI in the Repo
Tests, Proofs, and Validation Surface: What Is Actually Proved
Current Follow-On Queue: What Has Landed, What Is Active Now, and What Is Deliberately Deferred

Learn Frothy in 30 Minutes

This is the fast tour. The rest of the guide turns you into a maintainer, but this opening section is meant to get you reading and writing the current live Frothy surface immediately.

Scope note:

- The accepted `v0.1` semantics still matter, but this guide teaches the current prompt-facing surface that the shipped shell, tests, and starter materials accept today.
- Compatibility spellings such as `name(args)`, `=`, and `{ ... }` still show up in parts of the toolchain and parser fixtures. This guide does not use them as the primary teaching surface.
- Frothy is not a stack-first teaching language. If you know old Froth or Forth, treat that as substrate background, not as the user-facing product model.

One orientation before the examples:

- A **value** is a thing like `7`, `true`, `"tea"`, `nil`, a `Code` value, some `Cells`, or a current live record value.
- A **slot** is a stable top-level named place. The slot named `count` stays `count`; the value inside it can change.
- A **block** is code inside `[` and `]`.
- The REPL prints the value an expression returns. Mutations usually return `nil`.

Quick Syntax Map

These are the spellings to learn first:

- bind or rebind a top-level slot: `name is expr`
- mutate an existing place: `set place to expr`
- define named code: `to name [ ... ]` or `to name with a, b [ ... ]`
- create anonymous code: `fn [ ... ]` or `fn with a, b [ ... ]`
- call a named thing: `name: arg1, arg2`
- call computed code: `call expr with arg1, arg2`
- create local state: `here name is expr`
- define a fixed-layout record: `record Name [ field, ... ]`

- group names under a prefix: `in prefix [ ... ]`

Literals and direct values

```
42
true
false
"tea"
nil
```

> **RESULT**
>
> - `42`
> - `true`
> - `false`
> - `"tea"`
> - `nil`

Frothy starts from values you can see directly. There is no stack juggling to understand before you can read a line of code.

Stable top-level slots

```
count is 7
count
```

> **RESULT**
>
> - `count is 7` creates or rebinds the top-level slot named `count`
> - `count` evaluates to `7`

Think of `count` as a stable named box in the live image. You are not creating an ephemeral local that disappears at the end of the line.

Rebinding is central

```
message is "first"
to sayMessage [ message ]
sayMessage:
message is "second"
sayMessage:
```

> **RESULT**
> - the first `sayMessage:` returns `"first"`
> - the second `sayMessage:` returns `"second"`

That is one of Frothy's core ideas. Old callers observe the current value in the stable top-level slot. They do not capture a frozen copy.

Functions, anonymous code, and computed calls

Use `to` when you want to define a named top-level function directly:

```
to inc with x [ x + 1 ]
inc: 41
```

> **RESULT**
> - `inc` is now bound to a `Code` value
> - `inc: 41` returns `42`

Use `fn` when code is just another value:

```
double is fn with x [ x * 2 ]
double: 21
```

> **RESULT**
> - `double: 21` returns `42`

If the thing you are calling is itself produced by code, use `call`:

```
makeInc is fn [ fn with x [ x + 1 ] ]
call makeInc: with 41
```

> **RESULT**
> - `call makeInc: with 41` returns `42`

This is the common Frothy split:

- `name: ...` for a normal named call
- `call expr with ...` for a computed callee

Locals, mutation, and block results

Blocks use `[` and `]`. The last expression in the block is the result.

```
mathDemo is fn [
  here a is 20;
  here b is 22;
  a + b
]
mathDemo:
```

RESULT

- `mathDemo:` returns `42`

Use `set` when the place already exists and you want to change it:

```
count is 7
set count to 8
count
```

RESULT

- `set count to 8` returns `nil`
- `count` now returns `8`

Locals work the same way:

```
bump is fn [
  here n is 10;
  set n to n + 1;
  n
]
bump:
```

RESULT

- `bump:` returns `11`

Conditionals

Use `if` when you want both branches:

```
choose is fn with flag [
  if flag [ "yes" ] else [ "no" ]
]
choose: true
choose: false
```

> **RESULT**
>
> - `"yes"`
> - `"no"`

Use `when` for a one-branch conditional:

```
when true [ 42 ]
when false [ 42 ]
```

> **RESULT**
>
> - `42`
> - `nil`

Use `unless` for the opposite condition:

```
unless false [ 42 ]
unless true [ 42 ]
```

> **RESULT**
>
> - `42`
> - `nil`

Use `cond` when you want an ordered chain of conditions:

```
temperature is 72
threshold is 50
status is fn [
  cond [
    when temperature < 20 [ "cold" ];
    when temperature < threshold [ "ok" ];
    else [ "high" ]
  ]
]
status:
```

> **RESULT**
>
> - `status:` returns `"high"`

Use `case` when you want a switch-style dispatch over literal cases:

```
route is fn with mode [
  case mode [
    "off" [ 0 ];
    "on" [ 1 ];
    else [ 2 ]
  ]
]
route: "on"
```

RESULT

- `route: "on"` returns `1`

Important edge case:

```
flag is 1
if flag [ 1 ] else [ 2 ]
```

RESULT

- `flag is 1` succeeds
- the `if` fails with `eval error (3)`

Frothy does not use general truthiness. Conditions must evaluate to `Bool`.

Boolean short-circuiting

```
false and missingName
true or missingName
```

RESULT

- `false`
- `true`

This matters because it shows that the dangerous branch was not evaluated.

`repeat` and `while`

Use `repeat` for counted loops:

```
counter is cells(1)
set counter[0] to 0
repeat 5 as i [ set counter[0] to counter[0] + i ]
counter[0]
```

> **RESULT**
>
> - `counter[0]` is `10`

Use `while` when you want the lower-level loop:

```
sumTo is fn with limit [
  here total is 0;
  here i is 0;
  while i < limit [
    set total to total + i;
    set i to i + 1
  ];
  total
]
sumTo: 5
```

> **RESULT**
>
> - `sumTo: 5` returns `10`

That same pattern scales to larger checks:

```
sumTo: 100
```

> **RESULT**
>
> - `4950`

`Cells`: Frothy's narrow array value

`Cells` are the small mutable indexed store in Frothy's current core.

```
frame is cells(2)
set frame[0] to "left"
set frame[1] to 99
frame[0]
frame[1]
```

> **RESULT**
>
> - `"left"`
> - `99`

Today `Cells` are intentionally narrow. In the current live surface they are a good fit for:

- integers
- booleans

- `nil`
- text
- records

Important edge cases:

```
set frame[0] to fn [ 1 ]
frame[9]
```

> **RESULT**
>
> - storing `Code` fails with `eval error (3)`
> - out-of-bounds access fails with `eval error (13)`

That narrow policy is deliberate. `Cells` are Frothy's small mutable array, not its general "store anything" object bag.

Records

Records are now part of the live prompt-facing surface.

Define a record:

```
record Point [ x, y ]
```

Construct and read one:

```
point is Point: 10, 20
point->x
point->y
```

> **RESULT**
>
> - `10`
> - `20`

Mutate a field:

```
set point->x to 11
point->x
```

> **RESULT**
>
> - `11`

Records also work under prefixes:

```
in sprite [ record State [ x, y ]; current is State: 3, 4 ]
sprite.current->y
```

> **RESULT**
>
> - 4

Important edge case:

```
path is cells(2)
set path[0] to Point: 1, 2
path[0]->x
```

> **RESULT**
>
> - storing a record inside `Cells` succeeds
> - `path[0]->x` returns 1

Records are for shaped data. `Cells` remain the narrow array, and they still reject `Code`, other `Cells`, record definitions, and native values.

Prefix groups with `in`

Use `in prefix [ ... ]` to group related names under a shared prefix:

```
in math [
  to square with x [ x * x ];
  to cube with x [ x * x * x ]
]
math.square: 6
math.cube: 3
```

> **RESULT**
>
> - `math.square: 6` returns 36
> - `math.cube: 3` returns 27

This is the usual Frothy answer to “how do I make a tiny module or service namespace?”.

Inspection is part of normal work

First define something worth inspecting:

```
demo is fn [ when true [ 42 ] ]
```

Now inspect it:

```
words
show @demo
see @demo
core @demo
info @demo
info @save
```

What you should notice:

- `words` lists bound top-level names
- `show` and `see` print the normalized surface form
- `core` prints the canonical evaluator form
- `info @demo` shows an overlay, user-owned, persistable binding
- `info @save` shows a base, native, non-persistable binding

Inspection is not a last resort in Frothy. It is part of the normal workflow.

Persistence: `save`, `restore`, and `dangerous.wipe`

These are the image controls:

```
note is "draft"
save
set note to "edited"
restore
note
dangerous.wipe
note
```

> **RESULT**
>
> - after `restore`, `note` is back to `"draft"`
> - after `dangerous.wipe`, reading `note` raises `eval error (4)`

Records persist cleanly too:

```
record Counter [ value ]
counter is Counter: 0
to counter.bump [ set counter->value to counter->value + 1 ]
counter.bump:
counter->value
save
set counter->value to 9
restore
counter->value
```

> **RESULT**
> - after `counter.bump:`, `counter->value` is `1`
> - after `restore`, `counter->value` is back to `1`

What these commands mean:

- `save` snapshots the current overlay image
- `restore` replaces the live overlay with the saved one
- `dangerous.wipe` clears the live overlay and the saved snapshot

Boot hooks and safe boot

If a top-level slot named `boot` holds code, the runtime can run it at startup after restore.

One useful pattern is:

```
bootCount is 0
to boot [ set bootCount to bootCount + 1 ]
save
```

Then:

1. reset normally and check `bootCount`
2. reset again, but press `Ctrl-C` during the safe-boot window
3. check `bootCount` again

Expected:

- after a normal reset, `bootCount` is incremented
- after safe boot, the saved overlay is skipped and `bootCount` is undefined until you explicitly restore

One small board and FFI example

FFI means *foreign function interface*: the narrow bridge between Frothy values and native C or board code.

On the POSIX board, GPIO calls print traces so you can see what would happen on hardware. On a real ESP32 target, the same names hit the board driver.

```
led.on:
gpio.read: LED_BUILTIN
led.toggle:
gpio.read: LED_BUILTIN
adc.read: A0
adc.percent: A0
```

Typical POSIX result:

- LED helpers return `nil`
- `gpio.read: LED_BUILTIN` tracks the last written level
- `adc.read: A0` returns a deterministic integer such as `2048`
- `adc.percent: A0` returns a deterministic percentage such as `50`

Current board profiles may also expose:

- UART bindings such as `uart.init`, `uart.write`, `uart.read`
- I2C bindings such as `i2c.init`, `i2c.probe`, `i2c.write-reg`, `i2c.read-reg`
- constants such as `SDA` and `SCL`

If a binding appears in `words`, inspect it with `info @name` before using it.

Common patterns from other languages

Here are the Frothy equivalents of things people often look for first.

A small configuration service

```
sample.window is 16
sample.delay is 20
to sample.wait [ ms: sample.delay ]
```

Use stable top-level slots for configuration you expect to rebind live.

A switch statement

```
route is fn with mode [
  case mode [
    "idle" [ 0 ];
    "run" [ 1 ];
    "fault" [ 2 ];
    else [ -1 ]
  ]
]
```

Use `case` for literal-branch dispatch.

An if / else-if / else chain

```
classify is fn with n [
  cond [
    when n < 0 [ "negative" ];
    when n == 0 [ "zero" ];
    else [ "positive" ]
  ]
]
```

Use `cond` for ordered condition checks.

A struct with methods

```
record Counter [ value ]
counter is Counter: 0
to counter.bump [ set counter->value to counter->value + 1 ]
to counter.reset [ set counter->value to 0 ]
```

Use records for shaped data and prefixed helpers for behavior.

A namespace or module

```
in math [
  to square with x [ x * x ];
  to cube with x [ x * x * x ]
]
```

Use `in prefix [ ... ]` for grouped names.

A computed callback or dispatcher

```
apply is fn with f, x [ call f with x ]
apply: (fn with x [ x + 2 ]), 7
```

Use `call expr with ...` when the callee is computed.

A counter or accumulator

```
accumulate is fn with n [
  here total is 0;
  repeat n as i [ set total to total + i ];
  total
]
```

Use locals plus `repeat` or `while`.

Edge cases worth learning early

These are not obscure trivia. They are the rules that most often shape real Frothy code.

No truthiness

```
if 1 [ 1 ] else [ 2 ]
```

RESULT

- `eval error (3)`

No outer-local capture from nested `fn`

```
makeAdder is fn with bump [ fn with x [ x + bump ] ]
```

RESULT

- `parse error (108)`

If you want changing shared state, put it in top-level slots, records, or top-level-owned `Cells`.

Duplicate local bindings in one scope are rejected

```
bad is fn [ here x is 1; here x is 2 ]
```

RESULT

- `parse error (108)`

`set` only works on existing places

```
set missing to 1
```

RESULT

- `eval error (4)`

The prompt should stay healthy after errors

When you are unsure whether a failure is expected, the first question is: “Did the prompt recover cleanly?” If the answer is no, that is usually more serious than the original error.

Now that you can read and write current Frothy, here is how Frothy is actually built.

How to Read This Guide in Two Days

If you only have two or three days, do not read this like a reference manual. Read it like flight training.

Day 1:

1. Read the quickstart again, but type every example.
2. Read Part I through `Build and flash workflows`.
3. Run Lab 1 and Lab 2.
4. Re-read the chapters on stable slots, persistence, and inspection. Those are the three ideas that unlock the rest of the system.

Day 2:

1. Read Part II in order. The order matters: parser, then IR, then evaluator, then persistence, then control transport.
2. Keep one terminal open with the REPL and another with the codebase.
3. Read every `Code Walk` with the file open beside you.
4. Run Lab 3 if you have hardware.
5. Finish with Part III so the FFI and proof surface stop feeling magical.

If Thursday is close and you need the highest-value path, focus on these sentences until they feel obvious:

- Top-level slot identity is stable; values change, slot identity does not.
- `Code` is non-capturing in `v0.1`.
- Canonical IR, not raw source text, is the persisted truth for code.
- Persistence is overlay-only, not full execution-state persistence.
- The device image is the state. Host tooling is intentionally thin.

Part I: Using Frothy

Frothy's Core Mental Model and What Changed from Froth

Frothy is easier to understand once you stop asking "where did that stack item go?" and start asking "which place does this expression read or write?"

The core pieces are:

- **Values**: integers, booleans, `nil`, text, `Cells`, `Code`, and native bindings.
- **Places**: slots, locals, and indexed `Cells` positions.
- **Stable top-level slots**: named places that survive rebinding.
- **Locals**: short-lived places inside a running block or call.
- **Code**: non-capturing executable values.
- **Overlay image**: the mutable live layer on top of the base image.
- **Base image**: the built-in world the runtime seeds at boot, including `save`, `restore`, `dangerous.wipe`, inspection words, and filtered board bindings.

Old Froth let the visible stack model dominate how users thought about the system. Frothy deliberately moves the center of gravity. The substrate still exists underneath, especially in the board and FFI layers, but the language you maintain is lexical, named, and image-oriented.

Here is the picture that pays rent:

```
+-----------------------------+
|         Base Image          |
| save restore dangerous.wipe |
| words ...                   |
| filtered board slots        |
+--------------+--------------+
               |
               v
+-----------------------------+
|        Overlay Image        |
| your slots and rebound code |
| saved/restored/wiped        |
+--------------+--------------+
               |
               v
+-----------------------------+
|         Slot Table          |
| name -> stable slot id      |
| slot id -> current value    |
+-----------------------------+
```

The slot table is the stable naming layer. The overlay changes the current binding for a slot, but it does not invent a new slot identity each time you rebind.

Code Walk

Start with `src/frothy_base_image.c`.

`frothy_base_image_install()` seeds a fixed list of base slots: snapshot control, inspection words, and the special `boot` slot. It then calls `frothy_ffi_install_board_base_slots()` so the board surface becomes part of the same base world.

`frothy_base_image_reset()` is just as important. It does three jobs, in this order:

1. Release overlay-owned runtime state.
2. Clear any base slot bindings back to the built-in baseline.
3. Reset the heap pointer to the boot watermark and reinstall the base image.

That shape tells you something deep about Frothy's design: reset is not "try to undo the last few edits." Reset is "rebuild the base image and make the overlay earn its way back in."

Worked Example

```
message is "first"
to sayMessage [ message ]
sayMessage:
message is "second"
sayMessage:
```

Execution story:

1. `message is "first"` binds the stable slot `message` to a text value.
2. `to sayMessage [ message ]` stores `Code` in the stable slot `sayMessage`.
3. The first call reads slot `message` and returns `"first"`.
4. `message is "second"` writes a new value into the same slot.
5. The second call reads the same slot again and now returns `"second"`.

What to remember

- Frothy is an image language before it is a file language.
- Slots are stable top-level places.
- The overlay is the live, mutable layer.
- The base image is rebuilt, not patched in place, on reset and wipe.
- Old Froth substrate behavior matters only where Frothy intentionally reuses it.

Common confusions

- "Is `count is 8` creating a new variable?" No. It is writing a new value into the stable top-level slot `count`.
- "Does `Code` capture locals like a closure?" No, not in `v0.1`.
- "Is `dangerous.wipe` just clearing RAM?" Not exactly. It clears the overlay image and the saved snapshot, then rebuilds the base image.

Invariants

- A top-level slot's identity is stable once created.
- Base slots must come back after reset, restore failure, or wipe.
- Overlay bindings are the only bindings that persistence saves.
- A slot can change value class over time; the slot identity itself does not.

The Runtime Value Model: What a Value Is in Frothy

At runtime, every Frothy value fits in a 32-bit word. A **tagged value** is a word that carries both payload bits and enough tag information to tell you what kind of thing it is.

Some values are immediate:

- `Int`
- `Bool`
- `Nil`

Some values are references into the runtime's object table:

- `Text`
- `Cells`
- `Code`
- `Native`

The runtime structure in `src/frothy_value.h` makes the model concrete:

- an object table for live heap-like objects
- a payload arena for text and packed code bytes
- a `cellspace` region for `Cells`
- a scratch arena for temporary evaluation values

Diagram:

```
value word
  |
  +-- immediate tag -> int / bool / nil
  |
  +-- object-ref tag -> object table index
                          |
                          +-- TEXT   -> payload span in payload arena
                          +-- CELLS  -> span in cellspace
                          +-- CODE   -> packed IR payload + metadata
                          +-- NATIVE -> C function pointer + context
```

Why this shape?

- It keeps common values cheap.
- It keeps persisted objects pointer-free.
- It makes inspection and snapshotting tractable on small targets.

Code Walk

Read `src/frothy_value.h` and `src/frothy_value.c` together.

`frothy_value_t` is just a `uint32_t`, but the runtime struct around it is the real story. It preallocates bounded storage:

- `FROTHY_OBJECT_CAPACITY` objects
- `FROTHY_PAYLOAD_CAPACITY` bytes of payload
- `FROTHY_EVAL_VALUE_CAPACITY` temporary evaluation slots

This is not a general-purpose heap. It is a deliberately bounded runtime with a clear reset story.

`frothy_value_make_int()`, `frothy_value_make_bool()`, and `frothy_value_make_nil()` create immediates. `frothy_runtime_alloc_text()`, `frothy_runtime_alloc_cells()`, `frothy_runtime_alloc_code()`, and `frothy_runtime_alloc_native()` append live objects. `frothy_value_retain()` and `frothy_value_release()` then maintain object lifetimes with a **refcount** (reference count, meaning "how many owners still point at this object").

Worked Example

```
"frothy"
fn [ 1 ]
cells(1)
```

What happens conceptually:

- `"frothy"` creates a `Text` object backed by the payload arena.
- `fn [ 1 ]` creates a `Code` object backed by packed IR payload.
- `cells(1)` creates a `Cells` object backed by `cellspace`.

Those three values look equally ordinary at the language surface, but the runtime stores them in three different ways because their ownership and persistence stories are different.

What to remember

- Every value is a 32-bit runtime word.
- Some values are immediate; others are object references.
- `Text` and packed `Code` consume payload arena bytes.
- `Cells` consume `cellspace`, not the payload arena.
- Native bindings are values too, but they are not persistable.

Common confusions

- “Is `Code` stored as source text?” No. It is stored as canonical IR and associated metadata.
- “Are `Cells` just text or byte arrays?” No. They are their own object kind with narrow element rules.
- “Does `nil` allocate?” No.

Invariants

- Runtime values are always valid 32-bit tagged words.
- An object reference must point at a live object table entry.
- Releasing the last reference must clear the object and reclaim its storage.
- Persisted representations cannot depend on raw process pointers.

Stable Top-Level Slot Identity, Base Image vs Overlay Image, and Rebinding Semantics

The single most important semantic commitment in Frothy is that top-level names are stable slots, not throwaway dictionary entries.

That means:

- `count is 7` and `count is 8` talk about the same slot
- old callers observe new slot values
- snapshots save the overlay bindings for those slots
- `restore` rebuilds those bindings onto the same stable names

The base image versus overlay image split is how Frothy makes this practical.

- The **base image** is what boot always knows how to seed.
- The **overlay image** is what the user has changed since boot.

Code Walk

`src/frothy_eval.c` contains `frothy_slot_write_owned()`, the evaluator helper that turns a `WRITE_SLOT` IR node into a real slot mutation.

The function first looks up the slot by name. If the write is a creation-style write, it may create the slot. Then it releases any old bound value, marks the slot as overlay-owned, installs the new value, and updates the slot arity if the new value is `Code` or `Native`.

That last detail is easy to miss, but it matters: rebinding a slot is not only changing a value. It is also updating the calling surface the shell and control tools expect.

Worked Example

```
message is "first"
to sayMessage [ message ]
sayMessage:
message is "second"
sayMessage:
```

This example is worth repeating because it is the simplest proof that Frothy is late-bound at the slot level. The code stored in `sayMessage` does not freeze a copy of `message`. It performs a top-level slot read every time.

What to remember

- Slots are stable; values are replaceable.
- `is` is the create-or-rebind form.
- `set` is the mutate-an-existing-place form.
- Overlay writes are what snapshots save.

Common confusions

- "Why does old code see the new value?" Because top-level slot reads are late reads, not closure captures.
- "Does `restore` resurrect deleted local state?" No. It rebuilds top-level overlay state only.
- "Can a base slot become overlay?" Yes, by rebinding its value. The slot is still the same slot; the current binding becomes overlay-owned.

Invariants

- A slot name resolves to one stable slot table entry.
- Overlay writes must retain the installed value and release the old one.
- Rebinding must keep slot metadata, especially arity, truthful.
- Base image reset must be able to recover even after partial overlay failure.

Running Frothy in Practice

This chapter is the first hands-on loop. The goal is not to memorize commands. The goal is to make the image model feel real under your fingers.

Hands-on walkthrough

Lab 1: the local Frothy loop

Build the host runtime:

```
make build
```

Run it:

```
make run
```

You can also run the binary directly:

```
build/Frothy
```

A typical startup banner looks like this:

```
Frothy shell
snapshot: none
boot: CTRL-C for safe boot
Type help for commands.
frothy>
```

If you previously saved a snapshot, the second line may say `snapshot: found`.

Now do a clean local session:

```
dangerous.wipe
count is 7
record Point [ x, y ]
point is Point: 10, 20
to inc with x [ x + 1 ]
inc: 41
show @point
info @point
save
set point->x to 99
restore
point->x
```

What you should notice:

- `dangerous.wipe` gives you a known base-only image
- records are part of the current live prompt-facing surface
- inspection works on user-defined values, not just functions
- `restore` brings back the saved overlay exactly, not “something close”

Shell conveniences:

- `words` is the prompt-facing listing command
- `show @demo` and `see @demo` both route to the normalized binding view
- `core @demo` routes to the canonical core view
- `info @demo` routes to binding metadata
- `remember` routes to `save`

The prompt changes to `..` when the shell thinks your input is incomplete.

What can go wrong

- If the banner says `snapshot: found`, you are not starting from a blank image. Run `dangerous.wipe` first when teaching or debugging.
- `parse error (...)` means the parser could not build IR from the source.
- `eval error (...)` means parsing succeeded but execution failed, often because a name was missing or a type was wrong.
- `Ctrl-C` interrupts running evaluation and returns control at the next safe point.

What to remember

- The host runtime is not a toy. It is the main teaching and debugging target.
- `dangerous.wipe` is the best way to get back to a known state.
- Inspection is part of the normal workflow, not a last resort.

Frothy Projects in Practice

Frothy is an image language, but the CLI gives you a project scaffold so you can version source, dependencies, build settings, and board target choice cleanly.

Inside this repo, the repo-local CLI binary is `tools/cli/frothy-cli`. In a packaged release, the user-facing command is `frothy`.

Hands-on walkthrough

Lab 2: your first Frothy project

Create the default POSIX project:

```
tools/cli/frothy-cli new hello
```

That writes:

- `froth.toml`
- `src/main.froth`
- `lib/.gitkeep`
- `.gitignore`

The generated manifest looks like this in the important parts:

```
[project]
name = "hello"
version = "0.1.0"
entry = "src/main.froth"

[target]
board = "posix"
platform = "posix"

# [dependencies]
# utils = { path = "lib/utils.froth" }

# [ffi]
# sources = ["src/ffi/bindings.c"]
# includes = ["src/ffi"]
# defines = { MY_CONSTANT = "42" }
```

The current default POSIX scaffold still starts with a tiny compatibility-sugar boot example:

```
note = nil

boot {
  set note = "Hello from Frothy!"
}
```

That is still valid today, but it is not the primary teaching surface in this guide. The runtime meaning is the same Frothy image model you have been reading about.

The normalized equivalent, and the form to prefer in hand-written examples, is:

```
note is nil

to boot [
  set note to "Hello from Frothy!"
]
```

For the current workshop and hardware path, prefer the shipped demo-board flow instead of a project starter:

1. use the preflashed `esp32-devkit-v4-game-board`

2. open the exported `workshop/starter.frothy`
3. send or edit it live against the running board

That keeps the workshop path aligned with the actual shipped board state instead of introducing a second starter source.

`frothy send` does three important things for project source:

1. resolves includes
2. strips boundary markers
3. uses reset-plus-eval for whole-project load and then runs `boot:` or `autorun:` if present

Dependencies are source-level includes. A named dependency entry in `froth.toml` lets `\ #use "name"` resolve cleanly, but direct relative includes also work.

Useful project commands:

- `frothy send`
- `frothy build`
- `frothy connect`
- `frothy connect --local`
- `frothy doctor`
- `frothy tooling resolve-source`

What can go wrong

- The default POSIX scaffold still shows compatibility sugar. That does not mean the spoken surface is unsupported; it means the generated examples are not yet fully normalized.
- `frothy send` talks to a real device or runtime. If no device is connected, use `frothy connect --local` or the host binary directly.
- `frothy doctor` is the first thing to run when board setup feels uncertain.
- Include resolution errors usually mean the manifest dependency path or a relative `#use` path is wrong.
- Whole-file or project send is intentionally reset-plus-eval. If the connected firmware is too old to support control reset, fix the firmware instead of expecting additive replay to be safe.

What to remember

- `froth.toml` is the project authority.
- Project source is resolved and normalized before it is sent or built.

- The image remains the real state; project files are how you reconstruct it.

Build and Flash Workflows

The CLI supports three practical flows:

- POSIX project build
- ESP-IDF project build and flash
- prebuilt firmware flashing for the default ESP32 board

Hands-on walkthrough

What lands in `.froth-build/` for a POSIX project:

- `resolved.froth`: merged source before boundary stripping
- `runtime.frothy`: source that should be applied to the runtime image
- `firmware/Frothy`: the host executable
- `project_ffi.cmake`: generated only when `[ffi]` is present

For a normal POSIX project:

```
frothy build
frothy flash
```

`frothy flash` on POSIX is intentionally simple. It builds and then prints the binary path because there is no hardware flash step:

```
binary: /path/to/project/.froth-build/firmware/Frothy
```

For an ESP32 project:

```
frothy new --board esp32-devkit-v1 blink
cd blink
frothy doctor
frothy build
frothy flash
```

The important implementation detail is this: the CLI no longer tries to bake the whole runtime source into ESP-IDF firmware at build time. Instead it:

1. stages and builds the board firmware
2. flashes the firmware
3. reconnects

4. applies `.froth-build/runtime.frothy` into the live image

That is a better fit for Frothy's image model. Firmware and overlay state are related, but not the same thing.

Outside a project checkout, `frothy flash` now stops and tells you to use a Frothy project or repo checkout. Attendee boards are preflashed, and maintainer recovery stays source-based.

What can go wrong

- If `esp-idf` is missing, `frothy doctor` will tell you before `frothy build` fails later.
- If no USB-serial port is visible, `frothy flash` cannot choose a device.
- If several ports are visible, pass `--port`.
- On ESP32, build success does not mean your overlay source has been applied yet; that happens after flashing.

What to remember

- POSIX flash is a no-op that points at a binary.
- ESP32 flash is firmware first, runtime image second.
- `.froth-build/runtime.frothy` is the bridge between project source and live image state.

Part II: How Frothy Works

Memory, Ownership, Lifetimes, Refcounts, Capacities, Cellspace Usage, and Reset Stories

Frothy's memory model is small on purpose. You are maintaining a bounded live system, not a language runtime that assumes a desktop heap will quietly grow forever.

Important bounded capacities from `src/frothy_value.h` and `src/frothy_eval.c`:

- `FROTHY_EVAL_VALUE_CAPACITY 256`
- `FROTHY_OBJECT_CAPACITY 128`
- `FROTHY_PAYLOAD_CAPACITY 16384`
- `FROTHY_EVAL_FRAME_CAPACITY 128`

Those are compile-time limits. They are part of the design, not temporary scaffolding.

The first thing to internalize is that Frothy does not have one generic heap. It has several bounded stores with different jobs:

- the inherited substrate slot table stores stable top-level names and binding metadata
- the Frothy object table stores live heap-like objects such as `Text`, `Code`, `Cells`, record definitions, records, and natives
- the payload arena stores variable-sized object payloads such as text bytes, packed IR, record-def strings, and record field arrays
- `cellspace` stores the indexed contents behind `Cells`
- the evaluator scratch arena stores temporary locals and argument buffers
- the evaluator frame stack stores resumable execution frames for IR dispatch

The runtime value word itself is only 32 bits. From `src/frothy_value.c`:

```
bits ... 2 1 0
          ^^^^^ low tag bits

00 = signed 30-bit int
01 = special immediate
10 = slot designator
11 = object reference
```

Special immediates are:

- `nil = 0x1`
- `false = 0x5`
- `true = 0x9`

Integers are `value << 2`. Slot and object references are `index << 2 | tag`. That means a `frothy_value_t` is cheap to copy, cheap to store in `cellspace`, and still explicit about whether it names a slot or an object.

Runtime storage map

The important runtime structs are:

```
froth_vm_t
  slot table         -> inherited Froth substrate, stable name index
  cellspace          -> indexed cells storage + base seed/reset mark
  frothy_runtime_t
       object_storage[128]
       free_span_storage[128]
       payload_storage[16384]
       payload_free_span_storage[128]
       eval_value_storage[256]

src/frothy_eval.c
  frothy_eval_frame_stack[128]
```

`frothy_runtime_t` in `src/frothy_value.h` is the best single map of Frothy's owned memory.

The live object kinds are:

- `TEXT`: payload span + logical length
- `CELLS`: `base` and `length` into `cellspace`
- `CODE`: payload span + arity + local count + body node id + packed IR view
- `NATIVE`: C function pointer + context + exported name + arity
- `RECORD_DEF`: payload span containing field-count + record name + field names
- `RECORD`: payload span containing `frothy_value_t[field_count]` plus a retained pointer to its definition object

In other words, a `Cells` object is only a descriptor. The actual indexed data is in `froth_vm.cellspace.data`. A `Code` object is also mostly a descriptor. The actual cloned IR arrays live in the payload arena, and the embedded `frothy_ir_program_t` inside the object is just a view over that packed block.

Ownership and lifetime rules

Ownership works like this:

- top-level slots own the values bound into them
- local frames own the values stored in their local slots
- evaluator scratch buffers own temporary child results while a node is in progress
- object references use refcounts so shared values such as text, code, and records stay alive until the last owner releases them

The key retain/release entry points are `frothy_value_retain()` and `frothy_value_release()` in `src/frothy_value.c`.

When an object's refcount drops to zero, `frothy_runtime_clear_live_object()` releases the storage specific to that kind:

- `TEXT` returns its payload span to the payload free list
- `CELLS` releases any stored child values, zeroes the span, and returns the span to the cellspace free list
- `CODE` releases the packed IR payload span
- `RECORD_DEF` releases its packed names payload
- `RECORD` releases each field value, releases the retained definition, and releases the field-array payload

The free lists matter. Frothy does not just bump forever. The payload arena and cellspace both support span reuse and tail coalescing, so short-lived overlay objects do not permanently fragment the runtime.

`cellspace` versus payload arena

`src/froth_cellspace.h` and `src/froth_cellspace.c` define:

```
froth_cellspace_t
  data       -> active cells storage
  base_seed  -> saved base-image cells content
  used       -> next free cell index
  high_water -> max used seen
  capacity   -> total cells capacity
  base_mark  -> cutoff between base image and overlay cells
```

`cellspace` is for indexed `frothy_value_t` cells only. The payload arena is for bytes and packed structs. Keeping those stores separate is why `Cells` indexing is simple and why code/text/record payloads can be densely packed without pretending they are random-access cell arrays.

Reset story

`frothy_runtime_clear_overlay_state()` is the cleanup heart.

It does all of these, in order:

1. walk overlay slots and release their bound values
2. reset `cellspace` to the captured base image
3. discard every live object
4. clear free-span bookkeeping
5. zero payload-usage accounting
6. zero evaluator scratch usage
7. clear last FFI error bookkeeping
8. increment `reset_epoch`

That last field is what keeps reset safe while evaluation is running. Evaluator buffers and frame states remember the epoch they were allocated under. If a reset happens, later cleanup sees the epoch mismatch and avoids double-release on now-stale storage.

Worked Example

Run this in the REPL:

```
dangerous.wipe
texty is "hello"
codey is fn [ 1 ]
frame is cells(1)
save
dangerous.wipe
```

What gets released:

- the text payload for `texty`
- the packed code payload for `codey`
- the `Cells` span for `frame`
- the snapshot on storage because `dangerous.wipe` is both live reset and saved overlay wipe
- any overlay slot bindings that owned those values

What survives:

- the base image
- the ability to rebuild new overlay state from scratch

What to remember

- Frothy's memory model is several bounded stores, not one general heap.
- The runtime value word is tagged enough to tell scalar, slot, and object identities apart in 32 bits.
- Bounded capacities are part of the product shape.
- Refcounts are how object lifetimes stay explicit.
- `cellspace` is a separate arena used by `Cells`.
- The payload arena stores text, packed IR, and record payloads.
- Reset is a first-class operation, not an error-case hack.

Common confusions

- "Does Frothy have a garbage collector?" No. It uses reference counting plus explicit reset paths.
- "Is payload storage the same thing as `cellspace`?" No. Text and packed code live in the payload arena; `Cells` live in `cellspace`.
- "Why does evaluation have its own scratch arena?" To make temporary allocations bounded and easy to reset.
- "Where are locals stored?" In the evaluator scratch arena, not in object storage and not in the payload arena.

Invariants

- Every retained object must eventually be released or cleared by reset.
- `cellspace` must return to base on overlay reset.
- After overlay clear, object count, payload usage, and eval scratch usage must all be zeroed.
- `reset_epoch` must advance whenever overlay state is cleared.

Surface Syntax, Parser Structure, Lexical Resolution, and Why `Code` Is Non-Capturing

Frothy's current surface tries to read like spoken code:

- `count is 7`
- `set count to 8`
- `to inc with x [ x + 1 ]`
- `repeat 3 as i [ ... ]`

But the parser is not trying to preserve the exact source the user typed. Its job is to resolve names, lower sugar, and produce canonical IR.

Lexical resolution means "decide at parse time whether a name refers to a local or to a top-level slot." Frothy does that with scopes and frames in `src/frothy_parser.c`.

The parser state has three important dynamic arrays:

- `bindings`: visible local names and their assigned local indexes
- `scopes`: boundaries that say which bindings die when a block ends
- `frames`: one lexical function context, with its own local-index numbering and outer-capture policy

The crucial design choice is that `Code` is non-capturing in `v0.1`. In plain language: an inner function does not close over outer locals.

Why choose that?

- It keeps persistence straightforward.
- It keeps the evaluator simple.
- It preserves the "top-level slots are stable, locals are temporary" split.
- It avoids inventing a closure object model before the language needs one.

Code Walk

Read four regions in `src/frothy_parser.c`.

- `frothy_push_scope()` stores `binding_start`, so block cleanup can discard only the names that belong to that scope.
- `frothy_push_frame()` stores `scope_base`, `next_local_index`, and `reject_outer_capture`.
- `frothy_declare_local()` assigns the next local index in the current frame.

`frothy_resolve_name()` first searches locals in the current frame. If it sees a matching name in an outer frame and `reject_outer_capture` is true, it returns a signature error instead of silently creating a closure-like reference.

Then read `frothy_parse_fn_node()`. Every `fn` body pushes a frame with `reject_outer_capture = true`. That is the parser-level guardrail that keeps `Code` non-capturing.

Then read the lowerers:

- `frothy_parse_repeat()` lowers `repeat` into hidden locals plus `while`
- `frothy_make_short_circuit_node()` lowers `and` and `or` into `if`

- `frothy_parse_top_level_record()` lowers `record Name [ ... ]` into a top-level write of a `RECORD_DEF` node
- `frothy_parse_in()` sets `active_prefix`, which is why names inside `in led [ ... ]` become fallback slot reads and writes such as `read-slot-fallback "led.pin" "pin"`

This is a good example of Frothy's style. The rule is enforced early, not hand-waved as "undefined later."

Worked Example

This kind of pseudo-closure is not allowed:

```
to makeAdder with bump [
  fn with x [ x + bump ]
]
```

Why not?

- `bump` is a local of the outer function
- the inner `fn` would need to capture it
- `v0.1` intentionally rejects that model

So if you want reusable changing state, put it in a top-level slot or a top-level owned `Cells` object instead.

For a concrete lowering example, the parser fixture `tests/frothy_parser/fixtures/spoken_repeat_expr.ir` shows that:

```
repeat 2 as i [ i ]
```

becomes:

```
(seq
  (write-local 1 (lit 2))
  (write-local 2 (lit 0))
  (while
    (call (builtin "<") (read-local 2) (read-local 1))
    (seq
      (write-local 0 (read-local 2))
      (seq (read-local 0))
      (write-local 2 (call (builtin "+") (read-local 2) (lit 1))))))
```

That is the pattern to keep in mind: the parser prefers to widen the IR a little rather than add lots of special runtime opcodes.

What to remember

- The parser resolves names before evaluation.
- Current spoken-ledger surface is sugar over a smaller core.
- `Code` values do not capture outer locals in `v0.1`.
- `repeat`, `when`, and `unless` are parser-level conveniences, not special runtime object kinds.
- Prefix groups are mostly name rewriting plus fallback slot nodes, not modules or runtime namespaces.

Common confusions

- "If Frothy is lexical, why not closures yet?" Because lexical name resolution and closures are related but not identical. Frothy has the first without the second.
- "Does `fn` preserve my original source text?" No. It preserves canonical meaning.
- "Are top-level names resolved at parse time too?" The parser chooses slot-read versus local-read structure, but the actual slot value is read at call time.
- "Why are `record` and `cells(n)` top-level only?" Because the runtime and snapshot model are intentionally keeping collection ownership shallow.

Invariants

- A local name must resolve to one local index within the current frame.
- Outer-local capture from nested `fn` bodies must be rejected.
- Parser sugar must lower into valid core IR nodes.
- Parse errors must leave the REPL recoverable.

Canonical IR: Node Types, Literals, Links, Rendering, and Why IR Is the Persisted Code Truth

IR stands for *intermediate representation*: the tree form between parsing and evaluation. In Frothy, IR is not a private compiler detail that you can ignore. It is the canonical truth for code, inspection, and persisted `Code` objects.

`src/frothy_ir.h` gives you the real program container:

```
frothy_ir_program_t
  literals[]   -> ints, bools, nil, text
  nodes[]      -> fixed-size node headers/payloads
  links[]      -> variable-length edges and lists
  root         -> top-level root node id
  root_local_count
  storage_kind/storage_base/storage_size
```

The complete current node set is:

- `LIT`
- `READ_LOCAL`
- `WRITE_LOCAL`
- `READ_SLOT`
- `WRITE_SLOT`
- `READ_SLOT_FALLBACK`
- `WRITE_SLOT_FALLBACK`
- `SLOT_DESIGNATOR`
- `RECORD_DEF`
- `READ_INDEX`
- `WRITE_INDEX`
- `READ_FIELD`
- `WRITE_FIELD`
- `FN`
- `CALL`
- `IF`
- `WHILE`
- `SEQ`

That list tells you what the runtime really understands. If a surface feature is not in that list, it is sugar.

The `links[]` side table is the detail most people miss at first. Frothy does not embed variable-length child lists inside every node. Instead:

- `CALL` uses `first_arg` and `arg_count` into `links[]`
- `SEQ` uses `first_item` and `item_count` into `links[]`

- `RECORD_DEF` uses `first_field` and `field_count` into `links[]`, where each link points at a text literal id for a field name

That is why IR nodes stay fixed-size while still representing calls, sequences, and field lists.

Flow:

```
source text
   |
   v
parser
   |
   v
canonical IR
   |
   +--> surface renderer (`see`)
   |
   +--> core renderer (`core`)
   |
   +--> evaluator
   |
   +--> snapshot codec for persisted code
```

Why persist IR instead of source?

- normalized meaning is easier to restore than original spelling
- snapshots stay smaller and less ambiguous
- `see` and `core` can tell the truth about what will actually run
- later bytecode or caching can remain an additive optimization rather than the semantic source of truth

Code Walk

Read three regions in `src/frothy_ir.c`.

- `frothy_ir_render_surface_expr()` pattern-matches the IR and reconstructs a human-normalized surface form:
- a `WRITE_SLOT` becomes either `name is value` or `set name to value`
- a lowered repeat loop becomes `repeat ... as ... [ ... ]`
- an `if` without an else may render as `when` or `unless`
- `frothy_ir_render_code()` prints the exact core tree directly.
- `frothy_ir_program_clone_packed()` and `frothy_ir_program_init_packed_view()` explain how code objects store IR in the payload arena.

That packed layout is important. A dynamic parse uses separately allocated arrays and strings. When the evaluator creates a `Code` object, `frothy_runtime_alloc_code()` measures the program, allocates one aligned payload span, clones literals/nodes/links/string bytes into that one block, and then stores a `FROTHY_IR_STORAGE_VIEW` program that points into the payload arena. Releasing the code object releases the whole packed IR at once.

Worked Example

Given:

```
demo is fn [ when true [ 42 ] ]
```

`show @demo` prints:

```
to demo [ when true [ 42 ] ]
```

`core @demo` prints:

```
(fn arity=0 locals=0 (seq (if (lit true) (seq (lit 42)))))
```

That is not the runtime contradicting itself. It is the runtime showing you the same code at two useful levels:

- human-normalized surface form
- canonical evaluator form

For a second example, the record fixture `tests/frothy_parser/fixtures/record_def.ir` is:

```
(write-slot "Point" (record-def "Point" "x" "y"))
```

That tells you exactly how top-level record declarations live in the image:

- the slot `Point` is rebound
- the bound value is a `record-def` object
- record field names are data carried by that object, not metadata hidden somewhere else

What to remember

- Canonical IR is the code truth Frothy persists.
- Surface syntax can change without changing the deeper IR model.
- `see` is normalized source-like rendering.
- `core` is actual core structure.
- `links[]` is where variable-length child lists really live.
- Packed `Code` objects are views over one payload block, not loose malloc trees.

Common confusions

- "Why doesn't `see` preserve my exact spacing and wording?" Because it renders meaning, not original text.
- "If `repeat` exists in source, why is there no `REPEAT` IR node?" Because it lowers into locals plus `WHILE`.
- "Does persisting IR make the language less human?" No. It makes the running state more inspectable and less ambiguous.

Invariants

- Every persisted code object must decode into valid canonical IR.
- Renderer output must correspond to the same IR body, not to stale source text.
- Lowered sugar must be semantically faithful to the surface form.
- Packed IR views must point only inside the owning payload span.

Evaluator Architecture: Frames, Locals, Slot Lookup, Calls, Mutation, Loops, Interrupts, and Safe Points

The evaluator in `src/frothy_eval.c` is no longer the old recursive tree-walker. The current runtime executes canonical IR with an explicit frame stack and a dispatcher loop, which is exactly the architecture Frothy ADR-118 called for.

Important ideas:

- a **root local frame** is still allocated out of the evaluator scratch arena for one top-level evaluation
- an **evaluator frame state** is one resumable IR node execution
- **phase** is the resume point inside a compound node
- **call kind** says whether a call resolved to builtin, native, record-def constructor, or `Code`
- safe points are places where the dispatcher checks whether `Ctrl-C` interrupted the program

Code Walk

There are four especially important regions.

First, `frothy_frame_init()` allocates the root locals array out of the bounded `eval_value_storage`.

Second, `frothy_eval_frame_state_t` shows what a resumable node needs:

- `program`, `locals`, `local_count`, `node_id`, and `out`
- `phase` and `index`
- `values[3]` for fixed scratch intermediates
- `buffer` for variable-size arg/local storage
- `call_kind`, `target_program`, `target_node`, `native_fn`, and `native_context`
- `reset_epoch`

Third, `frothy_eval_node()` pushes one initial frame and then runs:

```
while exec.depth > 0:
  look at top frame
  poll interrupt at phase 0
  dispatch by node kind
  either:
    push child frame
    advance phase
    complete value and pop
    return error
```

Fourth, the helper state machines matter:

- `frothy_eval_call()` is a 4-phase call dispatcher
- `frothy_eval_if()` is a 3-phase conditional
- `frothy_eval_while()` loops by resetting its phase to `0` instead of growing call depth
- `frothy_eval_seq()` evaluates intermediate items into scratch slot `values[0]` and drops them before moving on

That is the current execution model. The runtime does not recurse IR-to-IR through the C stack anymore. It explicitly pushes child evaluator frames into `frothy_eval_frame_stack[FROTHY_EVAL_FRAME_CAPACITY]`.

The old guide description that called this a recursive tree-walker is now out of date.

Worked Example

```
counter is cells(1)
set counter[0] to 0
repeat 2 as i [
  repeat 3 as j [
    set counter[0] to counter[0] + i + j
  ]
]
counter[0]
```

Execution trace in ordinary language:

1. `counter` points at a one-cell `Cells` object.
2. The parser lowers each `repeat` into locals plus a `while`.
3. The top-level eval allocates one locals array from `eval_value_storage`.
4. `frothy_eval_node()` pushes an initial `SEQ` frame.
5. Each compound node pushes child frames instead of recursing in C.
6. Each `while` iteration returns its frame to phase `0`, polls interrupt, and reuses the same frame slot.
7. Each body iteration reads `counter[0]`, computes the new value, writes it back, and releases temporary child results.
8. The final result is `9`.

Calls are worth spelling out because they define most of the evaluator:

1. phase `0`: if the call is not a builtin, evaluate the callee
2. phase `1`: classify the callee as builtin, native, record-def, or `Code`
3. phase `2`: evaluate arguments left to right into `frame.buffer`
4. phase `3`: for `Code`, push the target code body as another evaluator frame

For `Code` calls, `frame.buffer` is sized to the callee's `local_count`, and the evaluated arguments occupy the first `arity` locals. That is how call-time local storage stays bounded and obvious.

What to remember

- The evaluator is an explicit dispatcher loop over bounded frame storage.
- Slot lookup is dynamic at call time.
- Safe points are explicit.
- Reset paths are deliberately folded back into normal REPL behavior.
- Hot execution does not allocate from the host heap; it uses bounded scratch arenas and fixed frame storage.

Common confusions

- "Why can old code see rebound top-level values?" Because slot reads happen at runtime, not at function-definition time.
- "Why does `repeat` seem like syntax magic?" Because it is parse-time sugar that becomes plain evaluator machinery.
- "Does `while` return the last loop body value?" No. It returns `nil`.

- “Why is there both `buffer.values` and `values[3]` in a frame?” The fixed array handles small fixed-arity intermediates cheaply; the buffer handles argument and local arrays whose size depends on the node or callee.

Invariants

- Locals must stay within the frame’s declared local count.
- Slot writes must release the old value and own the new value.
- Loop evaluation must poll for interrupts.
- Reset during evaluation must not leave stale temporary ownership behind.
- Frame-stack overflow must fail as `FROTH_ERROR_CALL_DEPTH`, not as a target crash.

`Cells`: Why They Are Narrow, What They Can Store, How Indexing Works, and Why They Are Top-Level Owned

`Cells` are intentionally narrow. That is not lack of imagination. It is a design decision that keeps the runtime, persistence, and mental model small.

In `v0.1`, `Cells` are the only collection value and they are constrained:

- created with `cells(n)`
- indexed with `frame[i]`
- mutated with `set frame[i] to value`
- intended to keep mutable state shallow rather than becoming a second general graph store

The stored value classes are deliberately restricted to:

- `Int`
- `Bool`
- `Nil`
- `Text`
- `Record`

Why so narrow?

- snapshot encoding stays simpler
- ownership stays easy to reason about
- you do not accidentally turn `Cells` into a second object graph language

Code Walk

Read three helpers in `src/frothy_eval.c`:

- `frothy_cells_value_allowed()`
- `frothy_read_index_owned()`
- `frothy_write_index_owned()`
- `frothy_runtime_alloc_cells()`

`frothy_cells_value_allowed()` is the policy gate. If you try to store `Code`, another `Cells`, a record definition, or a native binding, it returns `FROTH_ERROR_TYPE_MISMATCH`.

`frothy_read_index_owned()` resolves the `Cells` object, bounds-checks the index, reads the stored tagged value out of `cellspace`, and retains it.

`frothy_write_index_owned()` does the inverse carefully: validate the base, validate the index, validate the stored value class, release the old stored value, then replace it in `cellspace`.

`frothy_runtime_alloc_cells()` shows the actual storage story:

1. reuse a free span if one exists
2. otherwise call `froth_cellspace_allot()`
3. initialize each cell to `nil`
4. append a `FROTHY_OBJECT_CELLS` descriptor with `base` and `length`

So a `Cells` value is never a separately malloc'd array. It is an object-table descriptor pointing into `froth_vm.cellspace`.

Worked Example

```
frame is cells(2)
set frame[0] to "left"
record Point [ x, y ]
set frame[1] to Point: 7, 8
frame[0]
frame[1]->y
```

RESULT

- `"left"`
- `8`

This works because both stored values are allowed classes. If you try to store a `Code` value instead, the write fails.

What to remember

- `Cells` are narrow by design.
- Indexing is bounds-checked and type-checked.
- The cells payload lives in `cellspace`, not the payload arena.
- `Record` values are allowed in `Cells`; `Code`, `Cells`, natives, and record definitions are not.
- `cellspace` can reset back to a captured base image in one operation.

Common confusions

- "Why can't `Cells` hold code?" Because Frothy is not trying to be a general object graph language in `v0.1`.
- "Why do `Cells` exist at all if top-level slots are stable?" Because a stable slot often needs a little structured mutable state.
- "Are `Cells` persisted by pointer?" No. Snapshot restore rebuilds them by content.
- "Can `Cells` nest?" No. A cell may hold a `Record`, but not another `Cells` object.

Invariants

- `cells(n)` requires a positive size.
- Index reads and writes require an integer index in bounds.
- Stored values must be one of the allowed narrow classes.
- Replacing a cell value must release the old stored value.

Persistence: `save`, `restore`, `dangerous.wipe`, Snapshot Layout, Symbol/Object Tables, Pointer-Free Restore, and Failure Handling

Persistence is one of Frothy's defining traits, but it is deliberately scoped.

Frothy persists the overlay image only. It does **not** persist:

- the current call stack
- in-flight locals
- paused loop position
- arbitrary native process state

That is why the model stays robust.

Snapshot payload layout in plain language:

```
outer substrate snapshot header
  payload length
  CRC32
  A/B slot metadata

inner Frothy payload
  magic = FRTY
  snapshot version = 2
  IR version = 2
  symbol table
  object table
  binding table
```

The symbol table stores names. The object table stores persistable objects. The binding table says which overlay slots point at which values.

More concretely, `src/frothy_snapshot_codec.c` writes:

- symbols:
- `u32 symbol_count`
- repeated `u16 length + raw name bytes`
- objects:
- `u32 object_count`
- repeated `u8 kind + kind-specific payload`
- bindings:
- `u32 binding_count`
- repeated `u32 symbol_index + encoded value`

The encoded value tags are:

- `NIL`
- `FALSE`
- `TRUE`
- `INT`
- `OBJECT`

That last case is an index into the snapshot object table, not a process pointer.

Object payloads are kind-specific:

- `TEXT`: `u32 byte_length + raw bytes`

- `CELLS`: `u32 length + repeated restricted values`
- `CODE`: arity, local count, body, root, root local count, literal count, node count, link count, then literals, nodes, and links
- `RECORD_DEF`: field count, record name, then field names
- `RECORD`: definition object index, field count, then repeated restricted values
- `NATIVE`: rejected immediately as not persistable

The phrase **restricted value** matters. For cells payloads and record fields, the codec only allows:

- `Int`
- `Bool`
- `Nil`
- `Text`
- `Record`

That restriction is why `Cells` and records can persist without reopening the runtime into a general cyclic object graph.

Code Walk

Read `frothy_snapshot_restore()` in `src/frothy_snapshot.c`.

The restore path is deliberately conservative:

1. pick the active snapshot slot
2. read and parse the header
3. verify size bounds
4. read payload bytes
5. verify CRC
6. validate payload structure
7. reset to the base image
8. decode and rebuild the overlay image
9. on any failure, reset back to base again

That “reset back to base again” rule is what makes restore trustworthy. A bad snapshot should not leave you in half-restored limbo.

Then read three areas in `src/frothy_snapshot_codec.c`.

`frothy_snapshot_collect_overlay()` and `frothy_snapshot_collect_value_visit()` define the save walk:

- only overlay slots are roots
- names are interned into a symbol table
- reachable persistable objects are interned into an object table
- record cycles are rejected with a small `record_stack`
- code objects contribute slot symbols referenced from their IR

`frothy_snapshot_write_object_table()` defines the exact wire layout by object kind.

`frothy_snapshot_decode_payload()` shows the restore order:

1. decode symbols
2. decode objects into live runtime values
3. decode bindings and install them into slots

Code restore is especially worth reading. The decoder measures how much packed storage the restored IR will need, allocates one payload span, initializes a packed view into that storage, then fills the literals, nodes, links, and copied strings in place. Restored `Code` objects therefore end up in the same packed runtime shape as freshly evaluated `fn` values.

Worked Example

```
note is "draft"
frame is cells(1)
set frame[0] to 7
save
set note to "edited"
set frame[0] to 9
restore
note
frame[0]
dangerous.wipe
note
```

RESULT

- after `restore`, `note` is `"draft"`
- after `restore`, `frame[0]` is `7`
- after `dangerous.wipe`, `note` is undefined again

What to remember

- Persistence is overlay-only.
- Code is persisted as canonical IR, not as raw source text.
- Native bindings are non-persistable.
- Failed restore returns you to a clean base image.
- Symbol and object indexes replace process pointers in the payload.
- `Cells` and record fields use the restricted-value subset, which is a major part of why restore stays simple.

Common confusions

- "Does `save` persist everything running right now?" No. It persists the overlay image only.
- "Why does `dangerous.wipe` also erase the saved snapshot?" Because in Frothy it means factory-reset the user layer, not merely clear RAM.
- "If code is persisted, why does `see` still look nice?" Because `see` renders from canonical IR back into a human-normalized surface form.
- "Why is `record-def` both an IR node and an object kind?" The parser uses the node to express the declaration at top level; evaluation turns that node into a persistable record-definition object bound in a slot.

Invariants

- Snapshot payloads must validate before restore starts mutating live state.
- CRC mismatch must abort restore.
- Non-persistable values must make `save` fail instead of silently degrading.
- Restore failure must leave the system base-only and usable.
- Snapshot object indexes must always resolve within the decoded object table.

Inspection: `words`, `show`, `core`, `info`, and How Inspection Reflects Canonical IR Rather Than Raw Source

Frothy inspection is not decoration. It is part of the language contract.

The four core prompt-facing inspection tools answer different questions:

- `words`: what names currently have implementations?
- `show @name`: what is the normalized surface form of this binding?

- `core @name`: what is the canonical evaluator form?
- `info @name`: is it base or overlay, what class is it, and can it be persisted?

Code Walk

`src/frothy_inspect.c` keeps the implementation pleasingly literal.

`frothy_inspect_resolve_binding()` finds the slot, reads the bound value, figures out its class, and, if the value is `Code`, fetches the IR body, arity, and local count.

From there:

- `frothy_builtin_words()` prints all bound slot names
- `frothy_builtin_see()` prints a header and normalized surface rendering
- `frothy_builtin_core()` prints canonical IR
- `frothy_builtin_slot_info()` prints overlay/base, class, persistability, and ownership

The nice part is that the whole inspection surface is aligned with the runtime model you have already read. There is no second shadow representation.

Worked Example

```
demo is fn [ when true [ 42 ] ]
words
show @demo
core @demo
info @demo
info @save
```

What each line tells you:

- `words` confirms the name exists in the live image
- `show @demo` shows you the human-facing normalized form
- `core @demo` shows what the evaluator truly runs
- `info @demo` tells you it is overlay, code, persistable, user-owned
- `info @save` tells you it is base, native, non-persistable, foreign

What to remember

- Inspection reflects the canonical runtime truth.
- `see` is not a source file viewer.
- `core` is the fastest way to check what sugar lowered into.
- `info` is how you tell base/native from overlay/user state.

Common confusions

- "Why does `see` sometimes print `to ...` even if I used `fn`?" Because it renders the canonical binding view, not your original source spellings.
- "Why does `words` list board names?" Because filtered board bindings are installed into the base image like any other base slot.

Invariants

- Inspection of a missing name must fail cleanly.
- Code inspection must derive from canonical IR.
- Non-code bindings must still render truthfully as values or metadata.

Interactive Profile: Boot, Safe Boot, REPL, Multiline Input, Prompt Behavior, Ctrl-C, Recovery

Frothy is designed to be lived in interactively. The host shell is not a secondary toy wrapped around a compiler. It is the main user surface.

Boot flow:

```
main
  -> frothy_boot()
     -> platform_init()
     -> runtime init
     -> install base image
     -> print banner
     -> offer safe boot window
     -> maybe restore snapshot
     -> maybe run boot
     -> enter shell loop
```

Code Walk

Read `src/frothy_boot.c`.

`frothy_boot()` initializes platform and runtime state, installs the base image, prints the startup banner, offers a short `Ctrl-C` safe-boot window, restores a snapshot if one exists, runs `boot` if it is currently bound to `Code`, then enters `frothy_shell_run()`.

Two design choices are worth noticing.

First, safe boot happens before snapshot restore and `boot` execution. That means the recovery path is available even when user code or persisted state is broken.

Second, `frothy_boot_run_startup()` records restore and boot errors separately. That keeps start-up failures visible without destroying the ability to reach the prompt.

Then read `src/frothy_shell.c`. The shell has exactly the kind of niceties a small live language should have:

- `frothy>` prompt for complete input
- `..` continuation prompt for incomplete input
- shell command sugar like `show @name`
- `.control` mode entry

Worked Example

A typical manual recovery loop is:

1. start Frothy
2. press `Ctrl-C` during the safe-boot window if you suspect bad saved state
3. run `info @boot` or `show @boot`
4. run `dangerous.wipe` if the overlay needs a factory reset
5. reload or resend your project

That is a much better maintainer story than “delete files on disk and hope.”

What to remember

- Safe boot exists to protect the interactive workflow.
- `boot` is just a slot. It is special by convention and startup behavior, not because it lives in a separate language tier.
- Multiline input is part of normal use.

Common confusions

- “Why is `boot` listed even on a fresh system?” Because the slot is part of the base image, even when it has no user code bound into it.
- “Does `Ctrl-C` always kill the process?” No. In normal operation it is an interrupt signal for the running image.

Invariants

- Startup must always leave the user with a prompt or a clear hard failure.
- Safe boot must skip restore and boot execution.

- Shell command sugar must translate into valid Frothy source.

Control Transport: `.control` Mode, Single-Owner Sessions, Framed Requests/Events, Helper/Editor Story

The plain REPL is great for humans. Tooling needs something more structured. That is what `.control` mode is for.

The control channel is a framed request/event protocol over the underlying link. The key properties are:

- one owner at a time
- request/response sequencing
- output captured as structured events
- explicit idle event marking request completion

The transport underneath uses framed binary messages. The broader Froth tooling stack already uses **COBS** (*Consistent Overhead Byte Stuffing*, a byte-framing scheme that makes delimiter-based serial framing reliable), and Frothy rides on that substrate rather than reinventing it.

Request and event flow:

```
host tool
  |
  | AcquirePrompt() -> newline / Ctrl-C until "frothy> "
  | EnterControl()  -> ".control"
  v
device control mode
  |
  | HELLO / EVAL / WORDS / SEE / SAVE / RESTORE / WIPE / CORE / SLOT_INFO
  v
events
  |
  +-- OUTPUT
  +-- VALUE
  +-- ERROR
  +-- INTERRUPTED
  +-- IDLE
```

Code Walk

Read `src/frothy_control.h` first. The opcode list is the protocol summary:

- `HELLO`

- `EVAL`
- `WORDS`
- `SEE`
- `DETACH`
- `RESET`
- `SAVE`
- `RESTORE`
- `WIPE`
- `CORE`
- `SLOT_INFO`

Then read `frothy_control_start_capture()` and `frothy_control_send_output()` in `src/frothy_control.c`. They temporarily hook platform output so user-visible text becomes structured `OUTPUT` events attached to the active request sequence.

Finally, read `AcquirePrompt()` and `EnterControl()` in `tools/cli/internal/frothycontrol/session.go`. Those methods tell you how the host stays polite:

- first make sure the plain shell is really at a prompt
- then enter control mode
- then exchange framed requests until an `IDLE` event finishes the cycle

Worked Example

`frothy send`, `frothy build`, and `frothy flash` all depend on this machinery.

For example, after an ESP32 flash, the CLI reconnects and sends the staged runtime source through control mode rather than pretending firmware flashing and overlay loading are the same act.

That split is a good maintainer clue: the device image owns the state, and host tools should stay comparatively thin.

What to remember

- `.control` is the structured tool surface.
- Requests complete when the host sees `IDLE`, not merely when output stops.
- Control mode captures output and value separately.

Common confusions

- “Why not just scrape terminal text?” Because tools need explicit values, errors, and completion boundaries.
- “Can two hosts share the same control session?” No. The design is intentionally single-owner.

Invariants

- Session id and sequence id must match on both ends.
- Every handled request must end in `IDLE`, `ERROR`, or `INTERRUPTED`.
- Output capture must be detached cleanly after each request.

Part III: Extending Frothy

FFI Boundary: Native Bindings, Board Slots, Allowed Types, Non-Persistable State, Substrate Reuse

The FFI is where Frothy meets C and board code. It is intentionally narrow.

At the language surface, Frothy only lets native calls exchange a small value set cleanly:

- `Int`
- `Bool`
- `Nil`
- `Text`

It does **not** let the surface FFI freely pass:

- `Cells`
- `Code`
- native values

Why so strict?

- it keeps persistence sane
- it keeps the language model value-oriented
- it prevents board code from quietly widening Frothy into “whatever C can do”

Board bindings enter the base image as named native slots such as:

- `gpio.mode`
- `gpio.write`
- `ms`
- `adc.read`
- `uart.init`
- `uart.write`
- `uart.read`

Pins and constants like `LED_BUILTIN`, `UART_TX`, `UART_RX`, and `A0` arrive as base integer slots.

Code Walk

Read `frothy_ffi_dispatch()` in `src/frothy_ffi.c`.

That function is the translator between Frothy's value model and the inherited Froth substrate FFI. It:

1. validates argument count against the binding metadata
2. pushes Frothy values onto the Froth substrate stack in the expected form
3. calls the native word
4. converts zero or one result back into a Frothy value
5. restores the substrate stack depth even on failure

This is careful bridge code. It is not glamorous, but it is exactly where runtime corruption would begin if you got lazy.

Also notice the filtered name lists at the top of `src/frothy_ffi.c`. Frothy does not automatically expose every board binding that exists in board C code. The language surface is intentionally curated.

Worked Example

```
gpio.mode: LED_BUILTIN, 1
gpio.write: LED_BUILTIN, 1
ms: 50
gpio.write: LED_BUILTIN, 0
adc.read: A0
uart.init: UART_TX, UART_RX, 115200
```

On POSIX:

- GPIO calls print readable traces
- `adc.read` returns a deterministic test value
- `uart.init` returns a small integer handle

On hardware:

- the same names dispatch into the board implementation

What to remember

- FFI is value-oriented at the surface.
- Native values are not persistable.
- Board surface is filtered on purpose.

- Frothy reuses the older substrate here, but it does not inherit the old user-facing stack model.

Common confusions

- “If the board C file implements more names, why do I not see them?” Because Frothy exposes a filtered subset as base slots.
- “Can I save a native binding or native handle?” No. Native values are marked non-persistable.

Invariants

- FFI dispatch must restore substrate stack depth on success and failure.
- Surface FFI conversions must reject unsupported value classes.
- Base board names must be installed as base slots, not overlay slots.

Creating Project FFI in Practice

Project FFI is the extension point for “this project needs one small native thing in addition to the board baseline.”

The manifest surface is:

```
[ffi]
sources = ["src/ffi/bindings.c"]
includes = ["src/ffi"]
defines = { MY_CONSTANT = "42" }
```

And the expected C export is a null-terminated `frothy_ffi_entry_t[]` named `frothy_project_bindings`.

Hands-on walkthrough

Create the manifest section:

```
[ffi]
sources = ["src/ffi/bindings.c"]
includes = ["src/ffi"]
defines = { MY_CONSTANT = "42" }
```

Then create `src/ffi/bindings.c`:

```c
#include "frothy_ffi.h"

static froth_error_t project_magic(frothy_runtime_t *runtime,
                                   const void *context,
                                   const frothy_value_t *args,
                                   size_t arg_count,
                                   frothy_value_t *out) {
  (void)runtime;
  (void)context;
  (void)args;
  (void)arg_count;
  return frothy_ffi_return_int(MY_CONSTANT, out);
}

const frothy_ffi_entry_t frothy_project_bindings[] = {
    {
        .name = "project.magic",
        .params = NULL,
        .param_count = 0,
        .arity = 0,
        .result_type = FROTHY_FFI_VALUE_INT,
        .help = "Return a project-specific constant",
        .flags = FROTHY_FFI_FLAG_NONE,
        .callback = project_magic,
        .stack_effect = "( -- n )",
    },
    {0},
};
```

Then build:

```
frothy build
```

What build does today:

- validates that every `[ffi].sources` entry exists, is a `.c` file, and stays under the project root
- validates include directories
- validates define keys
- writes `.froth-build/project_ffi.cmake`
- passes `-DFROTH_PROJECT_FFI_CONFIG=...` into the build

What the generated CMake fragment contains:

- absolute source file paths
- absolute include paths
- compile definitions like `MY_CONSTANT=42`

Code Walk

Read `tools/cli/internal/project/ffi.go` and `tools/cli/cmd/build.go`.

`ResolveFFI()` is intentionally strict. It does not trust project paths by default. It resolves symlinks, rejects files outside the project root, and forces sources to be `.c` files.

Then `prepareProjectFFIConfig()` in `build.go` writes a tiny CMake fragment and the build passes that fragment to the top-level CMakeLists. This is a nice example of thin host tooling: the CLI does path validation and small build staging, then gets out of the way.

What can go wrong

- `[ffi]` without `sources` is rejected.
- source files that are missing, directories, or non-`.c` files are rejected.
- include paths must be directories under the project root.
- define keys must look like valid C identifiers.

Current-state note for maintainers:

The build plumbing and runtime auto-install path for `frothy_project_bindings` are both live in the maintained tree now. A project-local name such as `project.magic` should appear in `words()` once the project FFI table is compiled into the selected build.

The retained legacy `froth_project_bindings` export is still accepted as a compatibility bridge, but new code should start on the maintained `frothy_ffi_entry_t` path instead.

What to remember

- Project FFI is configured in `froth.toml`.
- The build path is strict and reproducible.
- Runtime installation is live; the maintainer job is to keep the exported binding table, source validation, and boot-time install path aligned.

Creating Board FFI in the Repo

Board FFI is the stronger, already-landed extension path. It is how a board becomes part of the base image.

Each board directory gives you three important artifacts:

- `board.json`
- `ffi.h`

- `ffi.c`

`board.json` defines board metadata and seeded pin constants. For example, the ESP32 DevKit V1 board declares pins such as `LED_BUILTIN`, `A0`, `UART_TX`, and `UART_RX`.

Hands-on walkthrough

A maintainer adding a new board-level capability goes through this flow:

1. add or adjust the native binding in `boards/<board>/ffi.c`
2. export it in the board's binding table
3. decide whether it belongs in the Frothy base-image surface
4. if yes, add its public name to the Frothy filter list in `src/frothy_ffi.c`
5. rebuild and check `words` plus a real call on POSIX or hardware

The filter step matters. Frothy deliberately ships a narrower board surface than the raw board substrate can expose.

Code Walk

Read three places together:

- `boards/esp32-devkit-v1/ffi.c`
- `boards/posix/ffi.c`
- `src/frothy_ffi.c`

The board files define many substrate-level bindings. Frothy then filters that set through `frothy_board_binding_names[]` and `frothy_board_pin_names[]` before installing names into the base image.

That means "implemented in board C" and "public in Frothy" are intentionally different stages.

This is a good maintainer pattern. It lets you keep extra board helpers or bring-up code around without accidentally widening the public language every time a board author adds a native word.

What can go wrong

- Adding a binding only in `boards/<board>/ffi.c` is not enough if you expect it to become a Frothy base slot.
- Adding too much to the public filter list widens the language surface and the persistence/inspection/testing burden that comes with it.
- Forgetting seeded pins in `board.json` makes board libraries and examples harder to read.

What to remember

- Board FFI is the landed extension path into the base image.
- Public surface is filtered on purpose.
- Seeded board constants are first-class readability tools, not mere metadata.

Tests, Proofs, and Validation Surface: What Is Actually Proved

The test suite is not just regression protection. It is executable teaching material.

High-value test files:

- `tests/frothy_parser_test.c`
- `tests/frothy_eval_test.c`
- `tests/frothy_snapshot_test.c`
- `tests/frothy_shell_test.c`
- `tests/frothy_ffi_test.c`
- `tools/cli/internal/frothycontrol/*.go` tests
- `tools/frothy/proof*.sh`

Code Walk

Read `test_spoken_ledger_surface_and_control_forms()` in `tests/frothy_eval_test.c`.

It is unusually valuable because it exercises the language the way a user thinks about it:

- spoken-ledger bindings and mutation
- `to` and `fn`
- locals
- `cells`
- `repeat`
- `when` and `unless`
- short-circuit boolean logic

Then read `tests/frothy_snapshot_test.c`. That file does more than check happy paths. It forces overflow, bad CRC, format errors, bad names, and failed restore reuse. That is the persistence contract being nailed down in executable form.

Finally, glance at the CLI proof scripts in `tools/frothy/proof*.sh`. Those prove end-to-end stories the unit tests alone cannot.

Worked Example

If you want one maintainer command that gives real confidence, run:

```
make test
```

What that actually buys you:

- parser coverage for surface forms and renderers
- evaluator coverage for value semantics and control flow
- snapshot coverage for save/restore/wipe and failure handling
- shell coverage for command sugar and prompt behavior
- FFI coverage for type conversion and filtering

What it does not magically prove:

- every board-specific hardware behavior on real devices
- every future syntax idea in draft docs
- the absence of all resource-limit edge cases on every target

What to remember

- Tests are part of the language explanation, not just guardrails.
- Snapshot failure-path tests matter as much as success-path tests.
- CLI and control tests prove the host/device boundary, not just C internals.

Common confusions

- “If tests pass, is the board workflow guaranteed on my machine?” Not necessarily. Toolchain and serial environment still matter.
- “Do proof scripts duplicate unit tests?” No. They cover end-to-end paths and release-shape behavior.

Invariants

- New semantics need new tests, not just new prose.
- Every recovery path worth trusting should be exercised somewhere.
- Proof surfaces must stay aligned with the shipped CLI and runtime.

Current Follow-On Queue: What Has Landed, What Is Active Now, and What Is Deliberately Deferred

The fastest way to stay honest as a maintainer is to separate three buckets in your head.

Landed and real today

- lexical named evaluation model
- stable top-level slots
- overlay-only persistence with `save`, `restore`, and `dangerous.wipe`
- canonical IR rendering and persistence
- REPL, safe boot, shell sugar, and `.control` mode
- records, prefix groups, `cond`, `case`, and `@` designators on the current live surface
- filtered board FFI surface, including GPIO, ADC, UART, and current board extras such as I2C where the profile exposes them
- project scaffolding, build, flash, doctor, and workshop editing workflows

Active now

- workshop operational closeout across clean-machine validation, room-side recovery prep, and one recorded workshop-board rehearsal

That is the immediate live priority because the major runtime and language tranches are already landed, and the remaining real risk sits in truthful operator proof on clean machines and real devices. The roadmap current-state block and `PROGRESS.md` are the authoritative control surface for that work.

Queued after the current operational closeout

- clean-machine validation on the promised attendee platforms
- classroom hardware and recovery kit closeout
- final measured workshop rehearsal
- post-workshop publishability reset tranches
- the bounded frame-arena ownership revisit if Frothy intentionally grows multiple live runtime instances again

Still deliberately deferred

- wider workspace/image-flow work beyond the first frozen doc tranche
- broader language widening beyond the landed surface

- runtime data models that would weaken persistence or inspection clarity

The healthiest maintainer posture is still this:

- ship the small system completely
- keep the invariants obvious
- only widen Frothy when the system feels stronger afterward, not merely larger